

Edge cases (200 doesn't mean safe to trust) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Story — `response_validation_practice.py`: a 200 status code only proves the transport worked — it says nothing about whether the body actually parses, or has the field your code expects. This exercise separates "the network succeeded" from "the content is usable," on purpose. **If not:** `http_client.py` would call `.json()` and index into the result directly, and the first time a server sends back an HTML error page with a 200 status, the whole client crashes with a confusing `JSONDecodeError` deep inside unrelated code.

Basic Version

Approach 1 — the direct way

```
# response_validation_practice.py
import json

# problem 1: invalid JSON
try:
    json.loads("<html>error page</html>")
except json.JSONDecodeError as e:
    print("bad json:", e)

# problem 2: valid JSON, missing key
data = {"choices": []}

value = data.get("message")
print("using .get():", value)  # None, no crash

try:
    value = data["message"]
except KeyError as e:
    print("using []: crashed with", e)
```

Expected output:

```
bad json: Expecting value: line 1 column 1 (char 0)
using .get(): None
using []: crashed with 'message'
```

This covers both problems correctly. It's missing type hints and doesn't explain *when* to pick `.get()` vs `[...]` in a comment — both fine for a first pass exploring the behavior.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — two named functions, so the choice is explicit

```
# response_validation_practice.py
import json
from typing import Any

class InvalidResponseBodyError(Exception):
    """Raised when an API response body cannot be parsed as JSON."""

def parse_body(raw_text: str) -> dict[str, Any]:
    """Parse a response body, raising a clear error if it isn't valid JSON."""
    # why: a 200 status only proves the transport worked – this is the
    # check that the body itself is actually usable JSON.
    # when: call this on every response body before touching any field in it.
    try:
        return json.loads(raw_text)
    except json.JSONDecodeError as e:
        # how: re-raises as a named error, with `from e` keeping the
        # original traceback attached instead of hiding what actually broke.
        raise InvalidResponseBodyError(
            f"response body is not valid JSON: {e}"
        ) from e

def get_required_field(data: dict[str, Any], key: str) -> Any:
    """Read a field the code can't work without. Fails loudly if missing."""
    # why: makes "this field must exist" an explicit, on-purpose choice
    # instead of an accidental KeyError buried somewhere later.
    # when: use this for a field your code cannot proceed without.
    if key not in data:
        raise KeyError(f"expected field '{key}' was missing from the response")
    return data[key]

def get_optional_field(
    data: dict[str, Any], key: str, default: Any = None
) -> Any:
```

```

"""Read a field that's genuinely optional – a missing value is normal."""
# when: use this for a field that's fine to be absent – the opposite
# case from get_required_field above.
# how: .get() with a default never raises, unlike data[key]
return data.get(key, default)

if __name__ == "__main__":
    try:
        parse_body("<html>error page</html>")
    except InvalidResponseBodyError as e:
        print(f"caught: {e}")

    payload = {"choices": []}
    optional = get_optional_field(payload, "message", default="no message")
    print("optional field:", optional)

    try:
        get_required_field(payload, "message")
    except KeyError as e:
        print(f"caught: {e}")

```

Expected output:

```

caught: response body is not valid JSON: Expecting value: line 1 column 1
(char 0)
optional field: no message
caught: "expected field 'message' was missing from the response"

```

(The first line is shown wrapped onto 2 lines just to fit the page — really one line of output.)

Difference from Basic: wrapping the two behaviors in named functions (`get_required_field` vs. `get_optional_field`) turns an implicit choice (`[key]` vs `.get(key)` , easy to pick inconsistently across a codebase) into an explicit, self-documenting decision every caller makes on purpose. `parse_body()` also re-raises `json.JSONDecodeError` as a named `InvalidResponseBodyError` with `from e` — this keeps the original traceback attached while giving callers one predictable, purpose-named error type to catch, instead of needing to know about `requests` ' or `json` 's specific exception classes (or a generic `ValueError` that could mean anything in a bigger codebase).

Which one should you actually write? For a response with 1-2 fields you care about, `get_required_field` / `get_optional_field` are enough, and are exactly what this document's Build Task uses. A response with a genuinely nested shape (most real LLM API responses have one)

is worth a `pydantic` model instead of hand-walking the path — Doc04's `chat_client.py` is where that switch actually happens.